

Выполнение действий для множества клиентов

Иногда возникают задачи, когда нужно выполнить какое-то действие для всех клиентов. Как правило, в этом случае делается вызов специального метода под служебным пользователем, а уже внутри производится обход схем.



Обратите внимание!

Этот подход работает только в рамках одного юнита. Для гарантированного выполнения операций необходимо использовать метод [CreateMultitenantEndpointByClientId](#).

Ранее это можно было сделать только с помощью метода `SetCurrentSearchPath()` ([C++](#) и [Python](#)), но у него есть существенный недостаток:

```
1 SetCurrentSearchPath('_00000003', public")
2 SqlQuery('select * from some_table') # тут все хорошо, данные
   получаютс из схемы нужного клиента
3 client_id = Session.ClientID() # тут вернется идентификатор служебного
   клиента, а не того, в рамках которого работаем
4 BLObject('SomeObject').AsyncInvoke('SomeMethod') # тут запрос уйдет
   под служебным пользователем, а не под "текущим" клиентом
```

Для решения этих проблем используется класс `TenantContext` (API в [C++](#) и [Python](#) (назван аналогично технологии sbis-multitenancy)), который принимает в конструкторе название схемы. На время жизни объекта этого класса подменяется сессия и меняется `search_path`, а при разрушении восстанавливается предыдущий контекст.

• Python

```
1 with TenantContext('_00000003'):
2     # do something
```

• C++

```
1 #include "sbis-bl-service/base_caller.hpp"
2 std::unique_ptr<bl::TenantContext> tenant_context =
   bl::GetCallerFactory().Create()->EnterTenantContext(L"_00000003");
```

Также в механизме `TenantContext` существует поддержка политик аутентификации. Вместо схемы можно передать одну из политик: `AuthByClientUserID`, `AuthByUserID`, `AuthByClientId`, `AuthByExtID` или `AuthByLogin`.

• Python

```
1 with TenantContext(AuthByLogin('some_user')):
2     # do something
```

• C++

```
1 #include "sbis-bl-service/base_caller.hpp"
2 #include "sbis-cloud-interaction/auth/user_id_auth_policy.hpp"
3 std::unique_ptr<bl::TenantContext> tenant_context =
   bl::GetCallerFactory().Create()->EnterTenantContext(
   cloud_inter::AuthByLogin( L"some_user" ) );
```